

2025-1

Basic Algorithm Study → For Beginners

기초 알고리즘 스터디 5주차

01

이분 탐색

· 검색을 해보자!

· 쿼리

02

매개변수 탐색

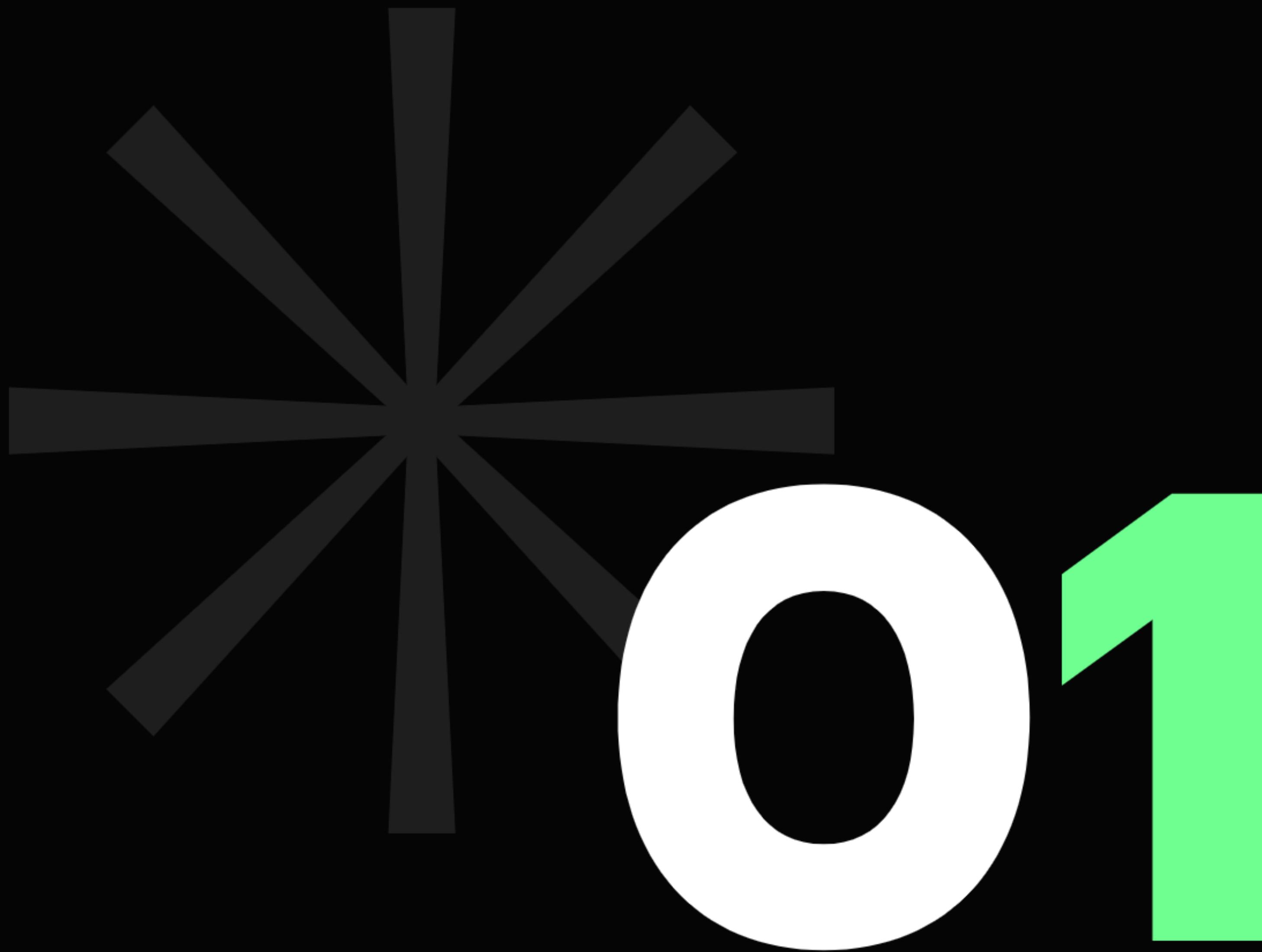
· 정의

· 어떤 때에 사용할까?

03

문제 풀어보기

이분 탐색

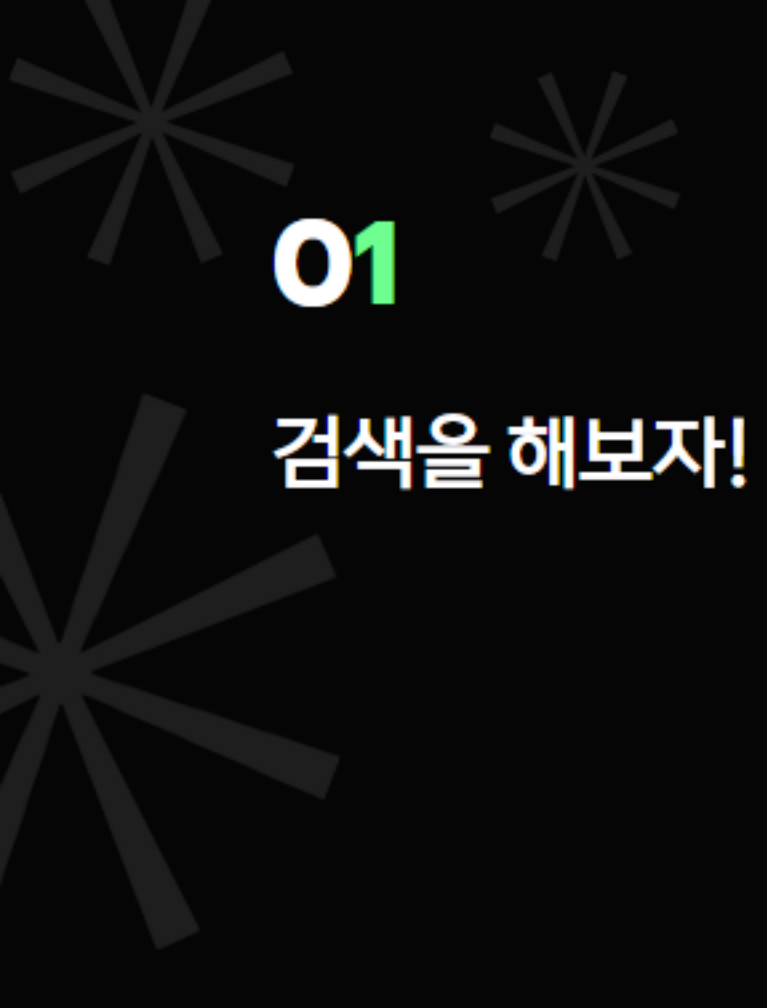




01

검색을 해보자!

예제: 수 찾기(#1920)



01

검색을 해보자!

10만개의 수를

길이 10만의 리스트를 돌면서 찾는다.

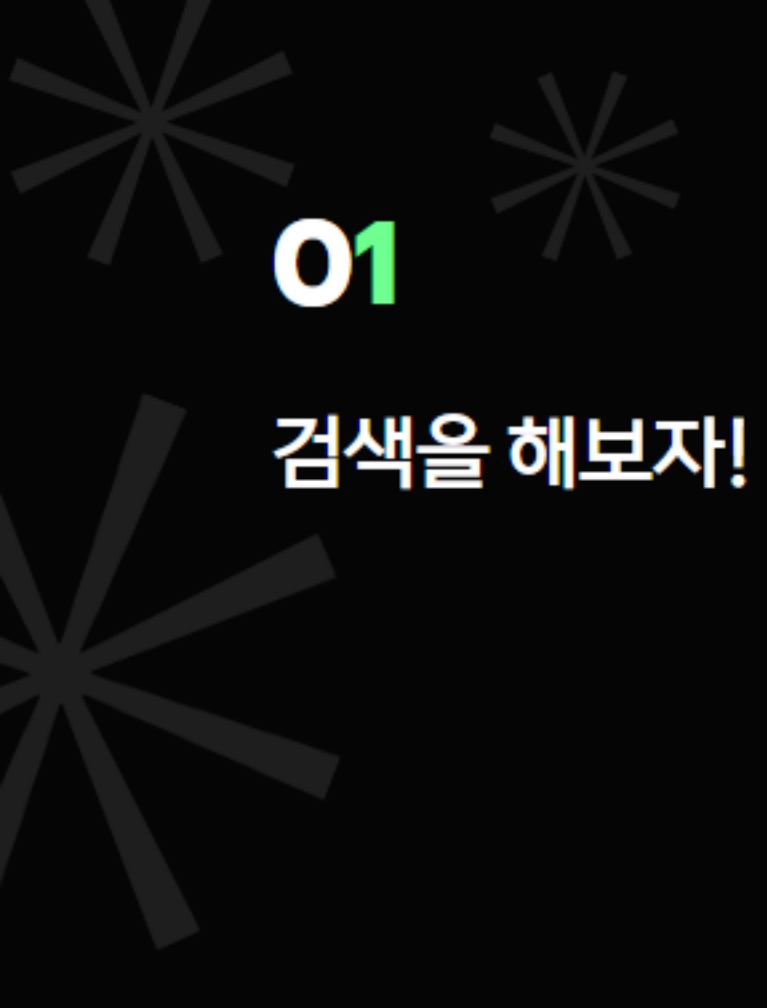
= $O(NM)$, $10^{10} = 10,000,000,000$



01

검색을 해보자!

하지만 이 문제는 난제가 아니다!
우선 입력받은 수들을 정렬해보자.



01

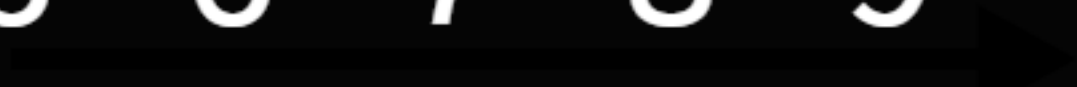
검색을 해보자!

정렬 알고리즘은 일반적으로 $O(N \log N)$
여기서 $\log$ 는 밑이 2인 로그이다.
 $2^{17} = 131,072 \sim 100,000$ 이므로
대략 170만회 정도로 수를 정렬할 수 있다.

01

검색을 해보자!

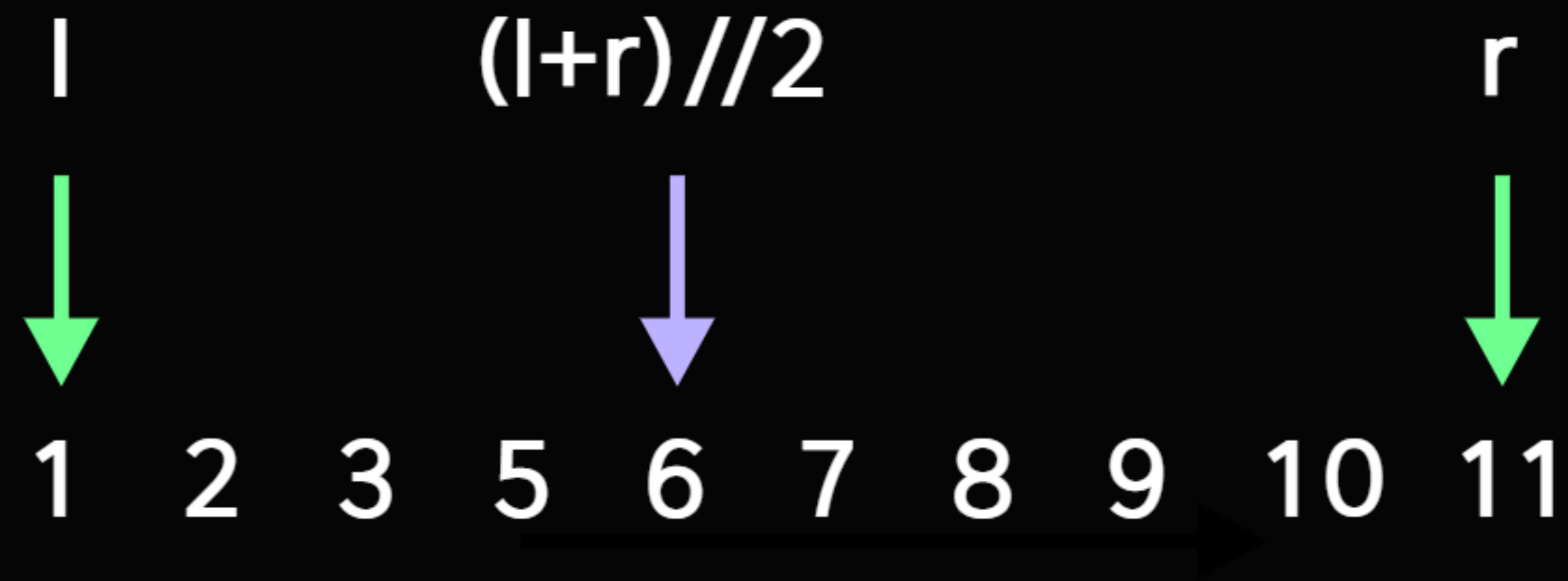
1 2 3 5 6 7 8 9 10 11



이 배열에서 3을 찾아보자.

01

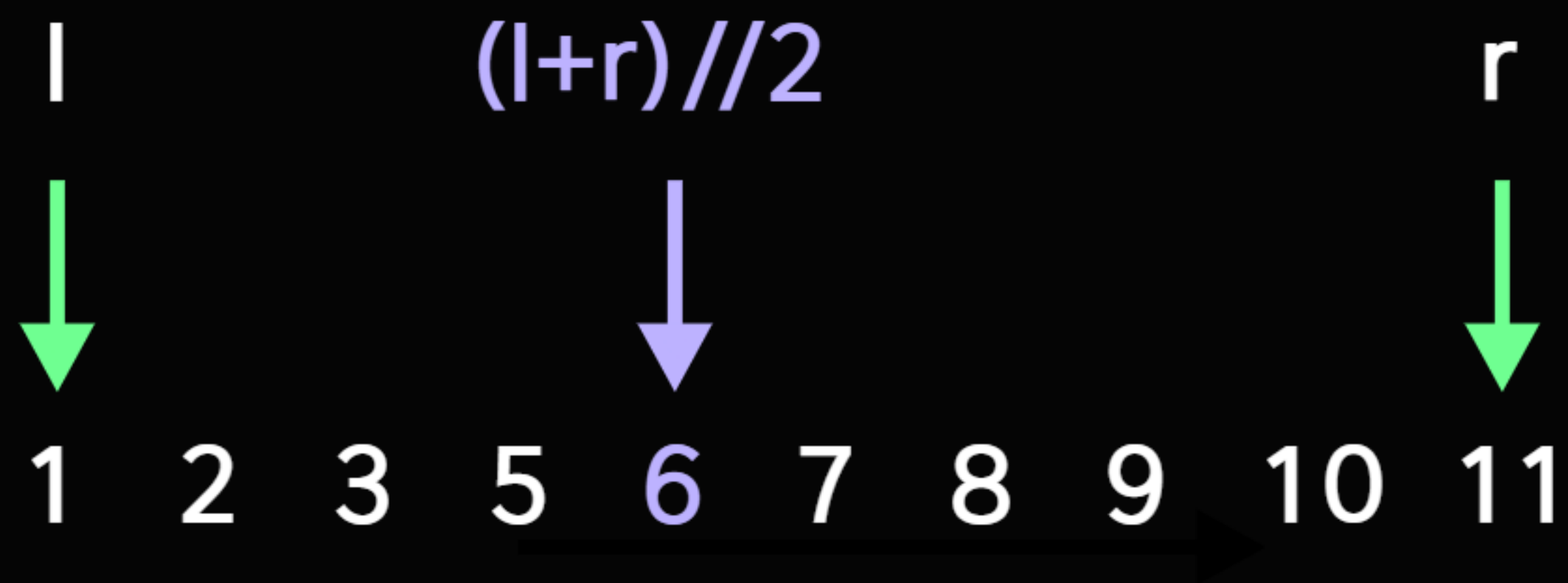
검색을 해보자!



이 배열에서 3을 찾아보자.

01

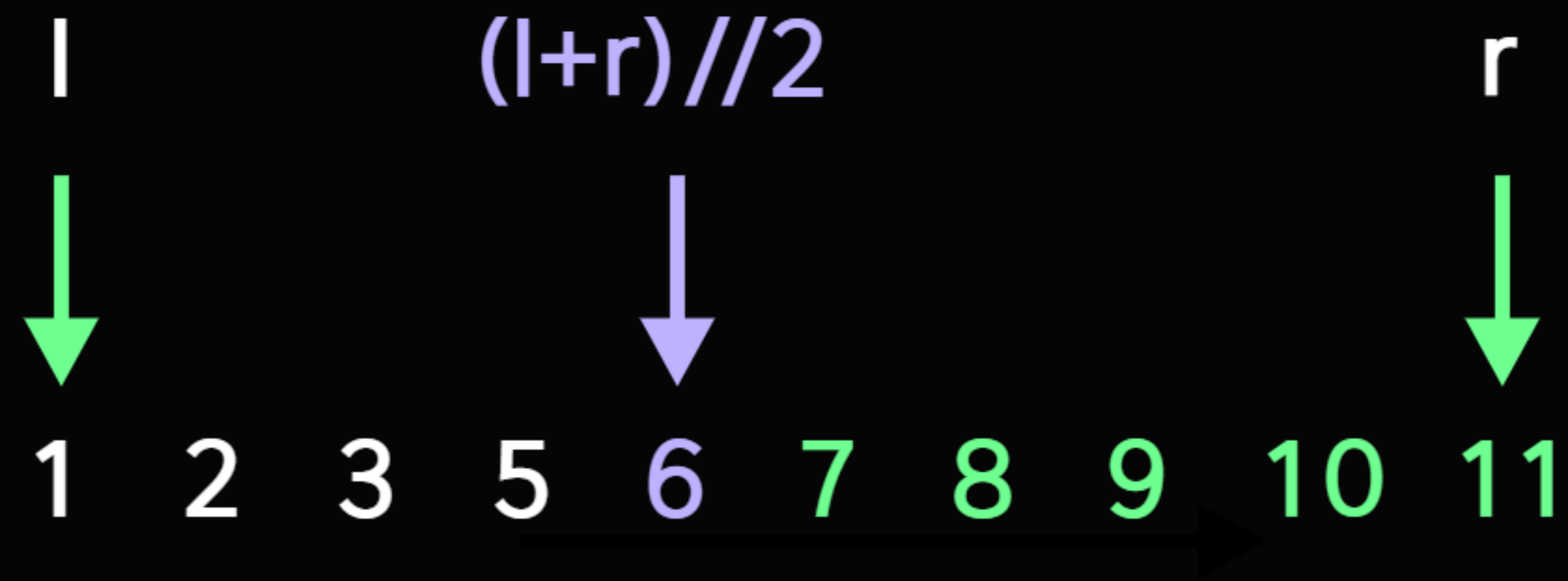
검색을 해보자!



중간 지점을 잡았더니,
찾으려고 하는 3보다 크다.

01

검색을 해보자!



그렇다면 지금보다 오른쪽에 있는 수들은
전부 3보다 클 것이다.

01

검색을 해보자!

l $(l+r)//2$ r



1 2 3 5 6 7 8 9 10 11

이제 찾으려는 수가 될 수 없는 범위는 전부 제외하고,
처음 과정을 반복하자.

오른쪽 탐색 범위를 6의 전으로 옮겨주면 된다.

처음에 비해 탐색 범위는 절반이 되었다.

01

검색을 해보자!

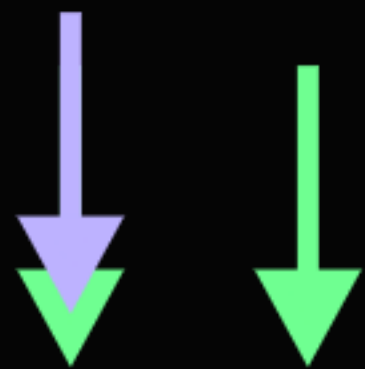


이번에 잡은 2는 찾으려는 3보다 작다.
그렇다면 2의 왼쪽에 있는 수들은 전부 3보다 작을 것이다.

01

검색을 해보자!

$$l = (l + r) // 2 \quad r$$



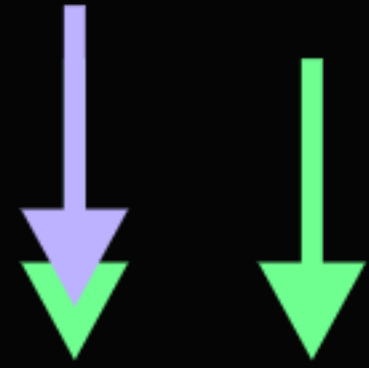
1 2 3 5 6 7 8 9 10 11

필요없는 범위를 버리기 위해
왼쪽 탐색 범위를 2의 오른쪽으로 옮겨주자.
중간 지점을 잡아보면 찾으려는 3이 나왔다!

01

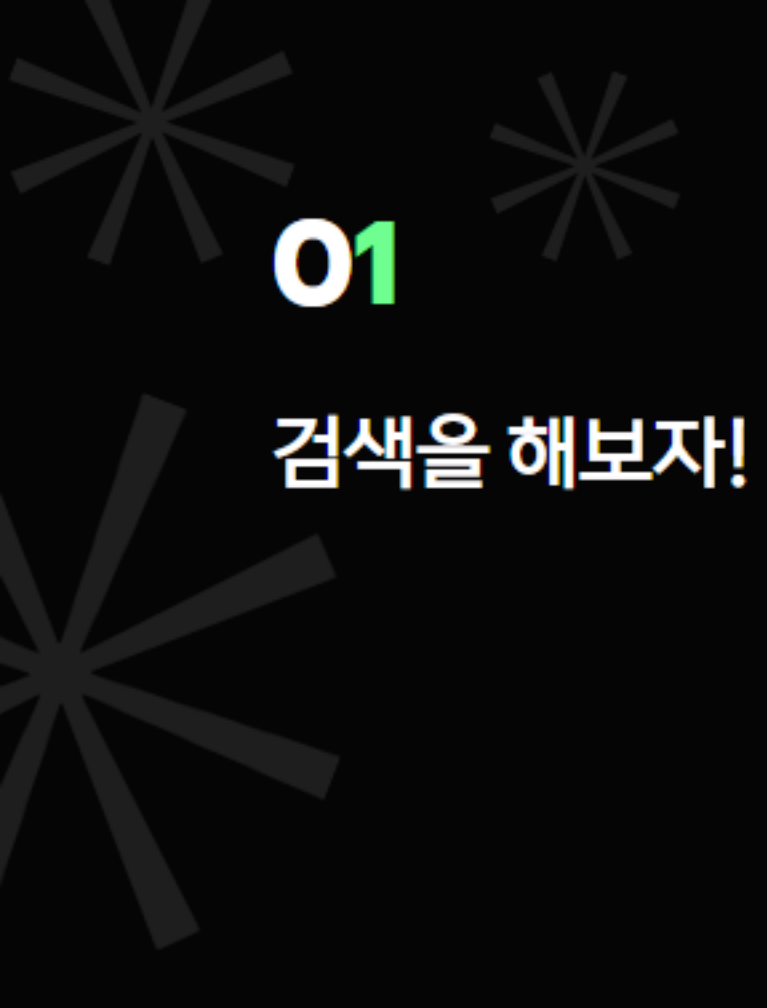
검색을 해보자!

$$l = (l + r) // 2 \quad r$$



1 2 3 5 6 7 8 9 10 11

반복을 멈추고 True를 반환한다.
이것이 이분 탐색이다.



01

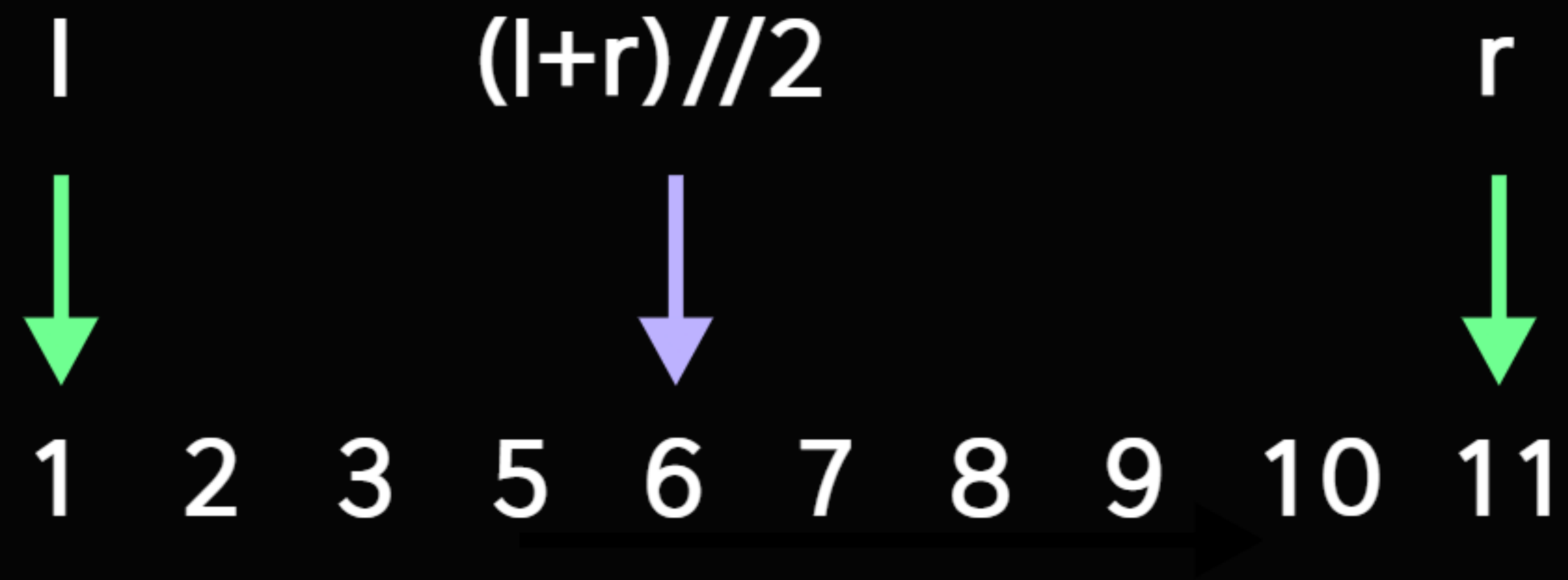
검색을 해보자!

이분 탐색은

정렬된 자료에서 원하는 값을 $O(\log N)$ 에 찾을 수 있다.
한 번 탐색할 때마다 탐색 범위가 반으로 줄어들기 때문에
 $\log_2(N)$ 안에 자료를 찾아낸다.

01

검색을 해보자!

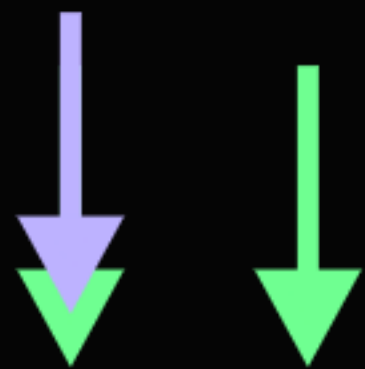


이번에는 배열에 없는 4를 찾아보자.

01

검색을 해보자!

$$l = (l + r) // 2 \quad r$$



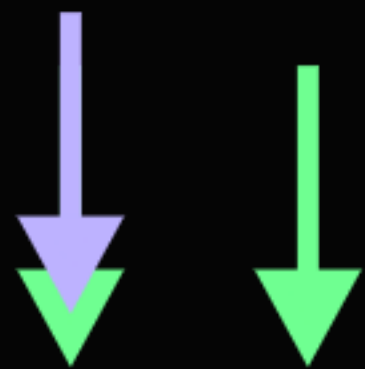
1 2 3 5 6 7 8 9 10 11

앞에서 했던 순서를 그대로 따라가면
여기까지는 모든 과정이 동일할 텐데...

01

검색을 해보자!

$$l = (l + r) // 2 \quad r$$



1 2 3 5 6 7 8 9 10 11

여기서 4는 3보다 크므로
3까지의 범위를 버려준다.

01

검색을 해보자!

$$l = (l+r)//2 = r$$



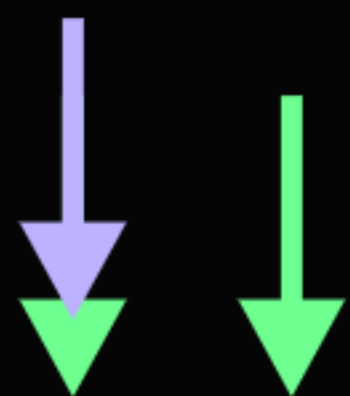
1 2 3 5 6 7 8 9 10 11

$l, r, (l+r)//2$ 가 전부 한 곳에 모였다.
하지만 이번에도 5는 찾으려는 값이 아니다.
찾으려는 4보다 크므로 ~5까지의 범위를 버려주자.

01

검색을 해보자!

$$r = (l + r) // 2 \quad l$$



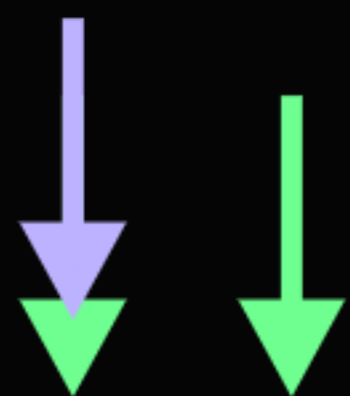
1 2 3 5 6 7 8 9 10 11

$l, r, (l+r)//2$ 가 전부 한 곳에 모였다.
하지만 이번에도 5는 찾으려는 값이 아니다.
찾으려는 4보다 크므로 ~5까지의 범위를 버려주자.

01

검색을 해보자!

$$r = (l + r) // 2 \quad l$$



1 2 3 5 6 7 8 9 10 11

이제 r 보다 l 이 더 큰 상황이다.

모든 탐색 범위를 버렸다는 말과 마찬가지로.

이렇게 되면 배열 안에 4가 없다고 판단하고 False를 반환한다.

이분 탐색으로 예제 (#1920)을 풀면 **시간복잡도**는
배열을 정렬하는 데 $O(N \log N)$,
검색 1회를 수행하는 데 $O(\log N)$.
M번 검색하므로 최종적으로는 **$O(M \log N + N \log N)$**
 $O(NM)$ 보다 훨씬 개선되었다.

N, M이 10만일 때
 $O(NM) = 100\text{억}$, $O(M \log N + N \log N) \leq 340\text{만}$

데이터가 주어지고, 수정이나 질문을 여러번 반복하는 문제를
쿼리(query)문제라고 한다.

이 문제가 만약 쿼리가 아니라 배열에서 한개의 값만 찾으면 되는 문제였다면
순차 탐색으로 $O(N)$ 에 해결하는 게 가장 빨랐겠지만,
쿼리 문제에서는 쿼리의 수를 고려해야 한다.

01

소스코드

```
import sys

input = sys.stdin.readline

N = int(input().strip())
lst = list(map(int, input().split()))
lst.sort()

M = int(input().strip())
target = list(map(int, input().split()))

def binarySearch(left, right, t):
    while left <= right:
        mid = (left+right)//2
        if lst[mid] == t:
            return 1
        if lst[mid] >= t:
            right = mid - 1
        else:
            left = mid + 1
    return 0

for t in target:
    print(binarySearch(0, len(lst)-1, t))
```

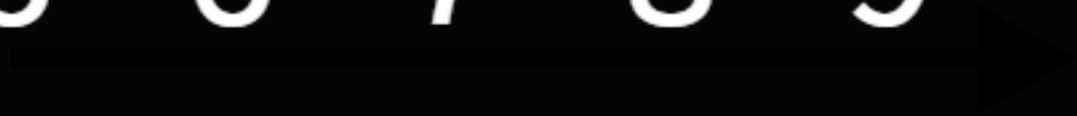
매개변수
탐색

02

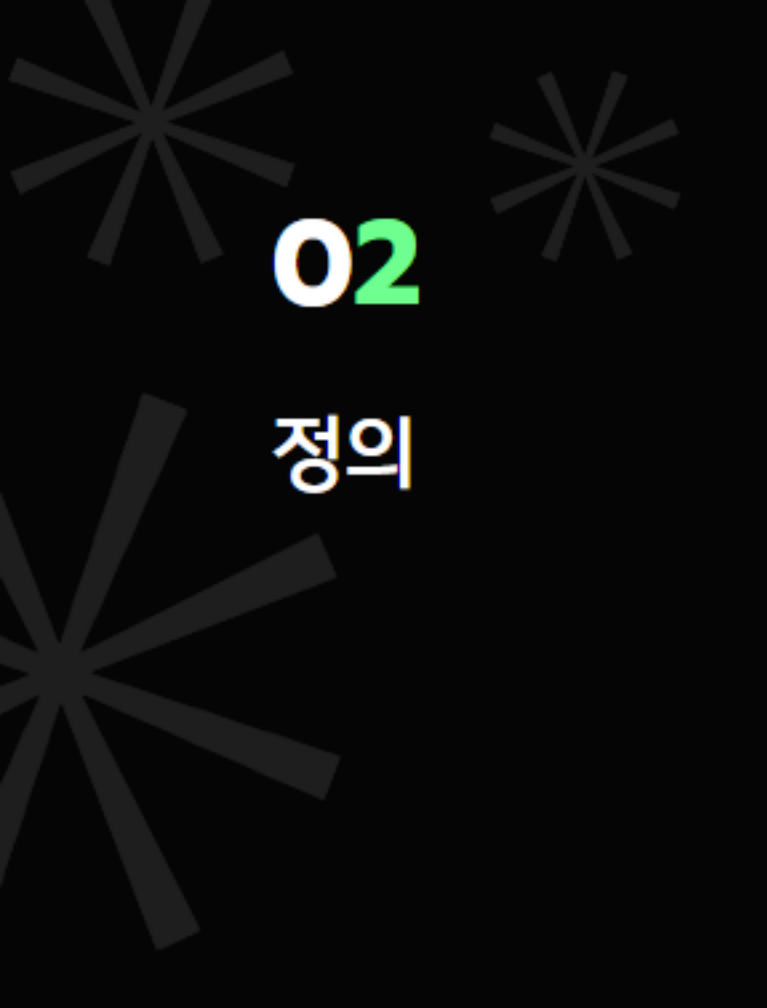
02

정의

1 2 3 5 6 7 8 9 10 11



생각을 바꿔서,
4보다 크거나 같은 수 중에서 가장 작은 수를 찾아보자.



02

정의

1 2 3 5 6 7 8 9 10 11

이 배열에서는 아마도 5가 나오겠지만...

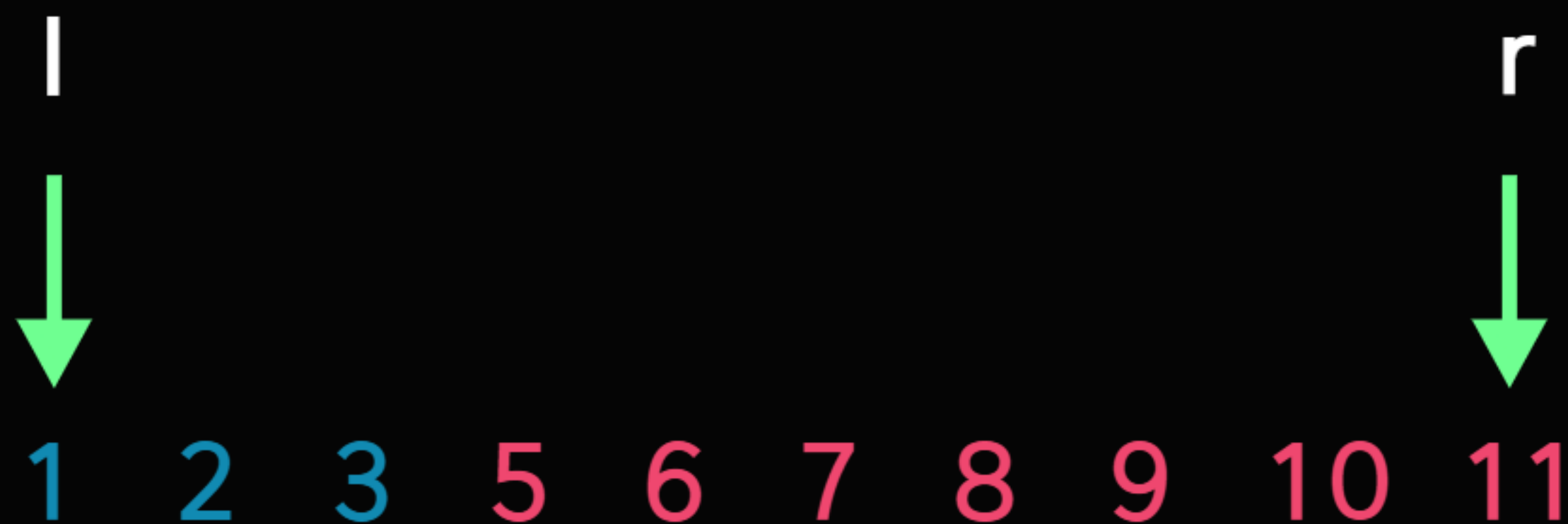
0번부터 차례차례 돌면서 처음으로 4보다 크거나 같아지는 수를 찾아볼까?

이 배열에서는 아마도 5가 나오겠지만...
0번부터 차례차례 돌면서 처음으로 4보다 크거나 같아지는 수를 찾아볼까?

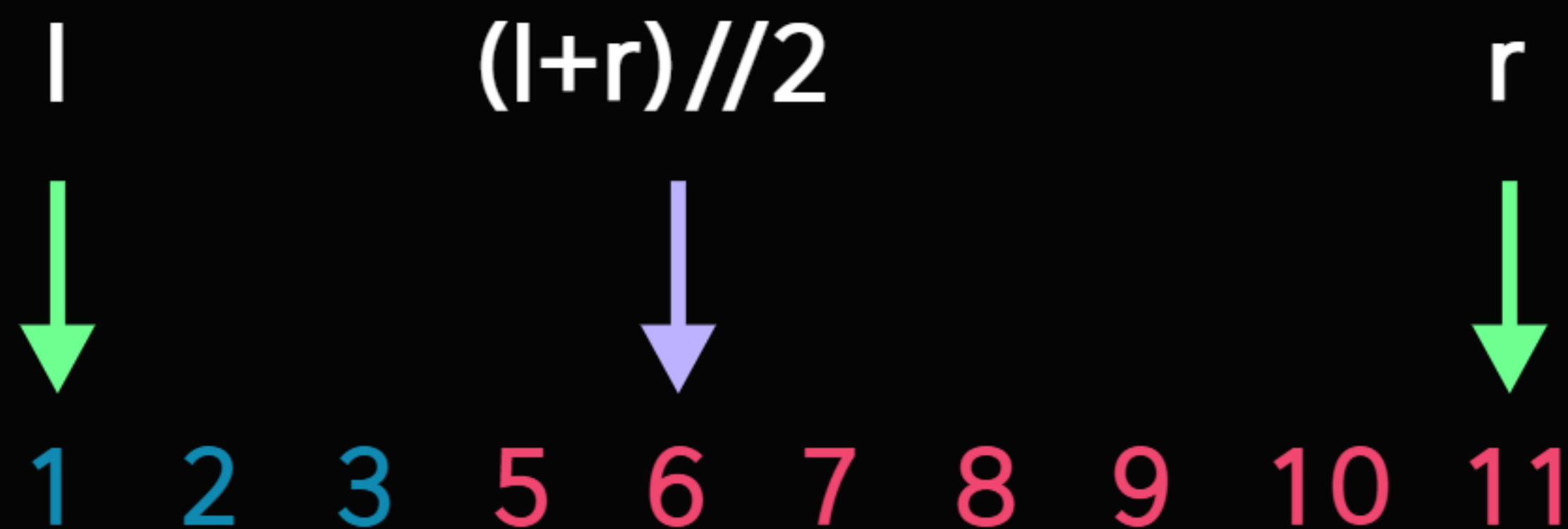
이 방식으로 찾는다면
순차 탐색이므로 시간복잡도는 $O(N)$, 선형 시간일 것이다.

02

정의



놀랍게도 정렬된 상태에서는 $O(N)$ 보다 더 빠른 방법이 있다.
우선 l, r 을 각각 4보다 작은 상태, 4보다 크거나 같은 상태로 정의하자.



그리고 이분 탐색을 할 때 처럼

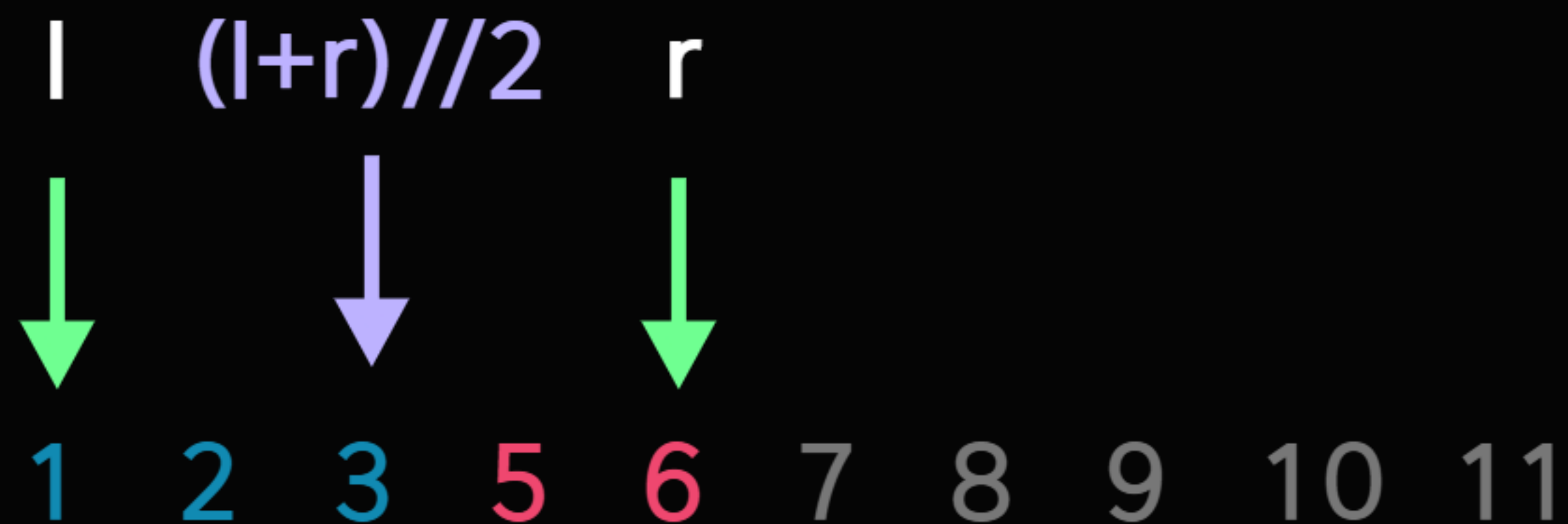
중간을 찍어서 4보다 작은지 판별해 보자.

이번에는 4보다 크거나 같으므로, r 을 중간으로 옮겨준다.

$r \sim \max$ 구간에는 절대 우리가 찾는 경계가 없음이 증명되었기 때문이다.

02

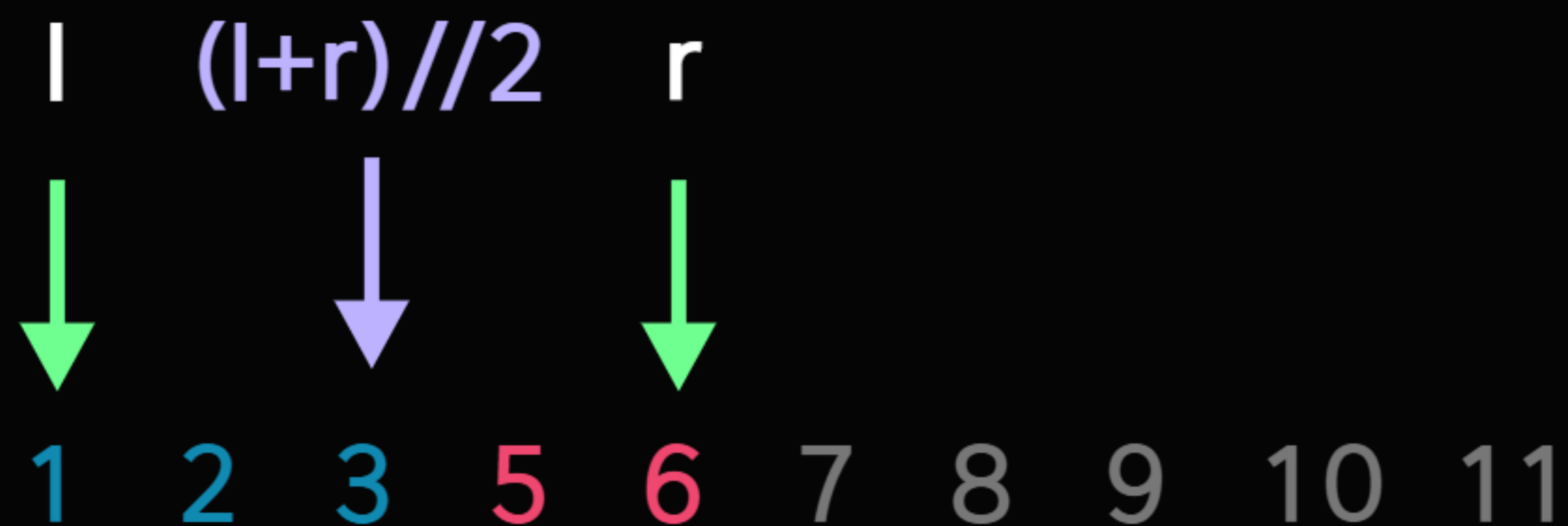
정의



이때 여전히 처음 만든 l, r 의 정의를 위배하지 않는다.
 l 은 4보다 작은 상태, r 은 4보다 크거나 같은 상태
하지만 범위는 반으로 줄었다!

02

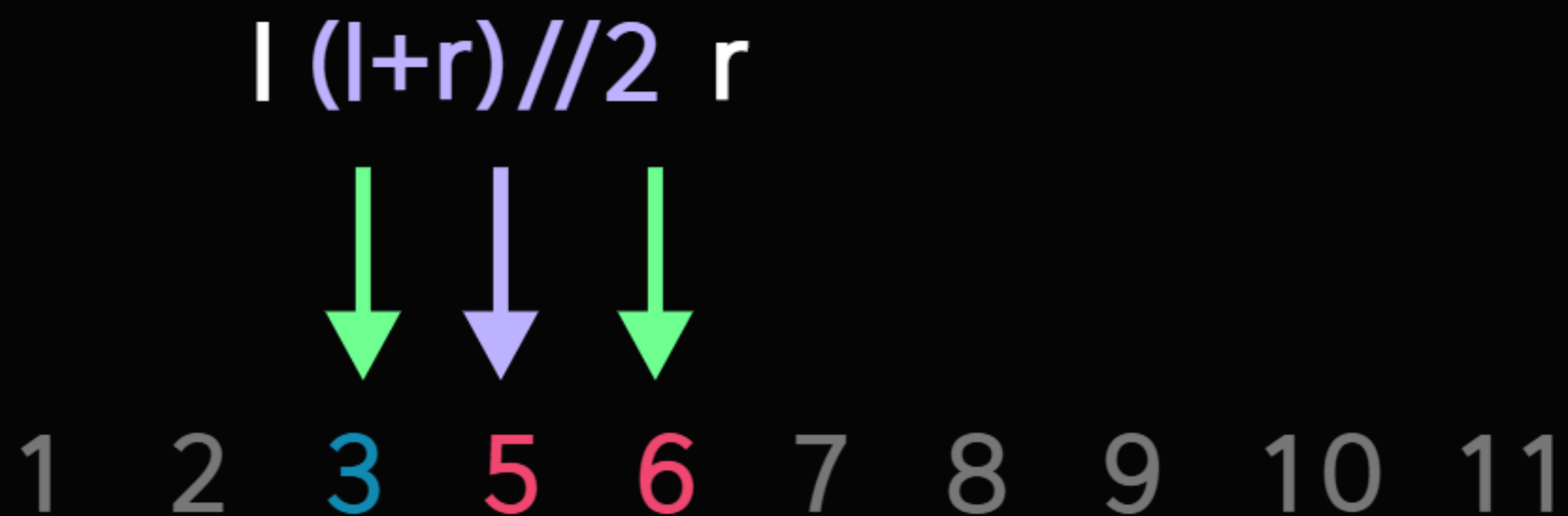
정의



이번에 찍은 3은 4보다 작으므로
l의 정의와 맞는다.
l을 3으로 옮겨주자.

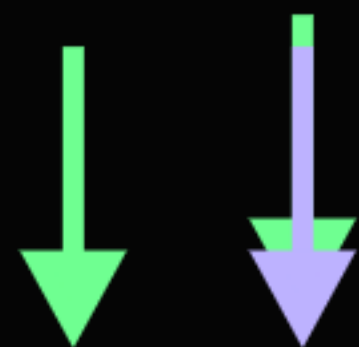
02

정의



다시 중간을 잡는다.
5는 4보다 크므로 r을 중간으로 옮겨준다.

$$l + (l+r)//2 = r$$



1 2 3 5 6 7 8 9 10 11

이제 l 과 r 이 한칸 차이로 나란히 있게 되었다.
이때 l 은 4보다 작은 상태, r 은 4보다 크거나 같은 상태이다.
경계를 찾았으니 이제 반복을 멈추고 r 을 반환한다.

12보다 작은 수

1 2 3 5 6 7 8 9 10 11

0보다 큰 수

1 2 3 5 6 7 8 9 10 11

〈주의사항〉

이 방법은 주어진 범위 내에 경계가 존재할 경우에만 답을 찾을 수 있다.
위와 같은 경우는 예외처리를 하거나 더미 데이터를 추가해야 한다.



02

어떤 때에 사용할까?

예제: 풍선 공장(#15810)

02

어떤 때에 사용할까?



최소공배수..? 함수..? 수열..?



02

어떤 때에 사용할까?

반대로 생각해보자.

주어진 시간이 T 일 때 만들어지는 풍선의 수를 구하는 문제라면?



02

어떤 때에 사용할까?

스태프 i 가 T 초동안 만드는 풍선의 수는
 $T//A_i$



02

어떤 때에 사용할까?

스태프 i 가 T 초동안 만드는 풍선의 수는
 $T//A_i$

N 명을 돌면서 한번씩 몫을 구하면 되니까 $O(N)$

T의 값이 적절한지 아닌지 판별하는 방법은 알았다.
그렇다면 적절한 T의 값을 어떻게 찾을까?

아하! 아까 배운 매개변수 탐색을 사용해보자.

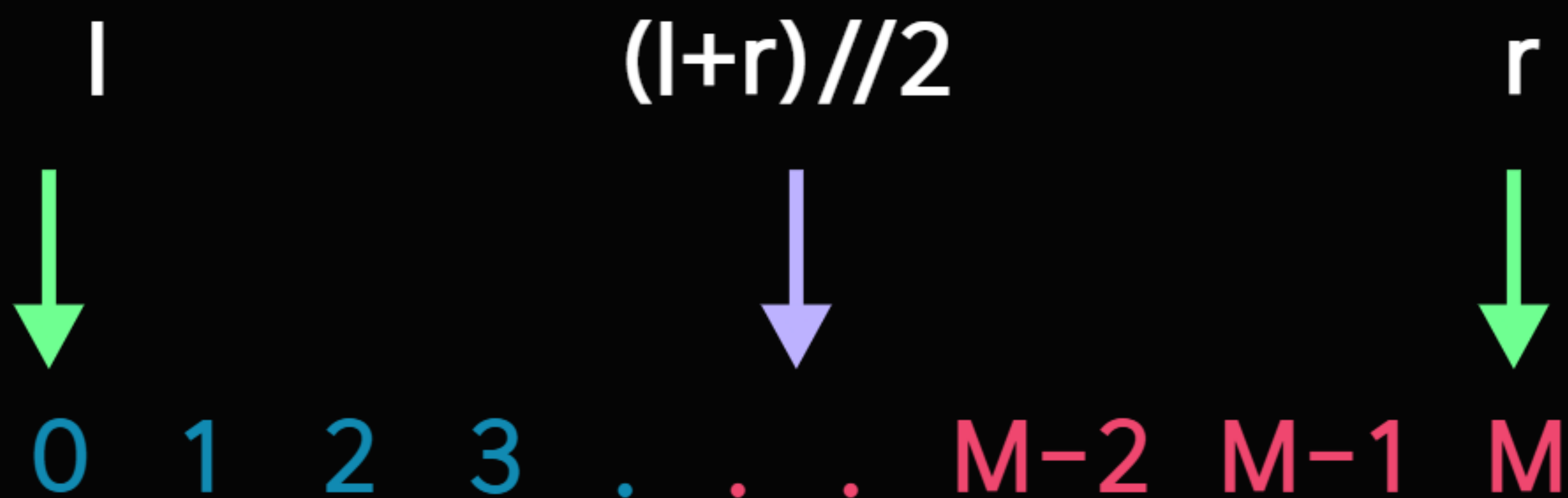
$A = [1, 2, 3]$ 인 경우를 생각해보자.

0 1 2 3 . . . M-2 M-1 M

$T=0$ 이라면 당연히 M개의 풍선을 만드는 것은 불가능하다.
반대로 $T=M$ 이라면 반드시 M개 이상의 풍선을 만들 수 있다.

02

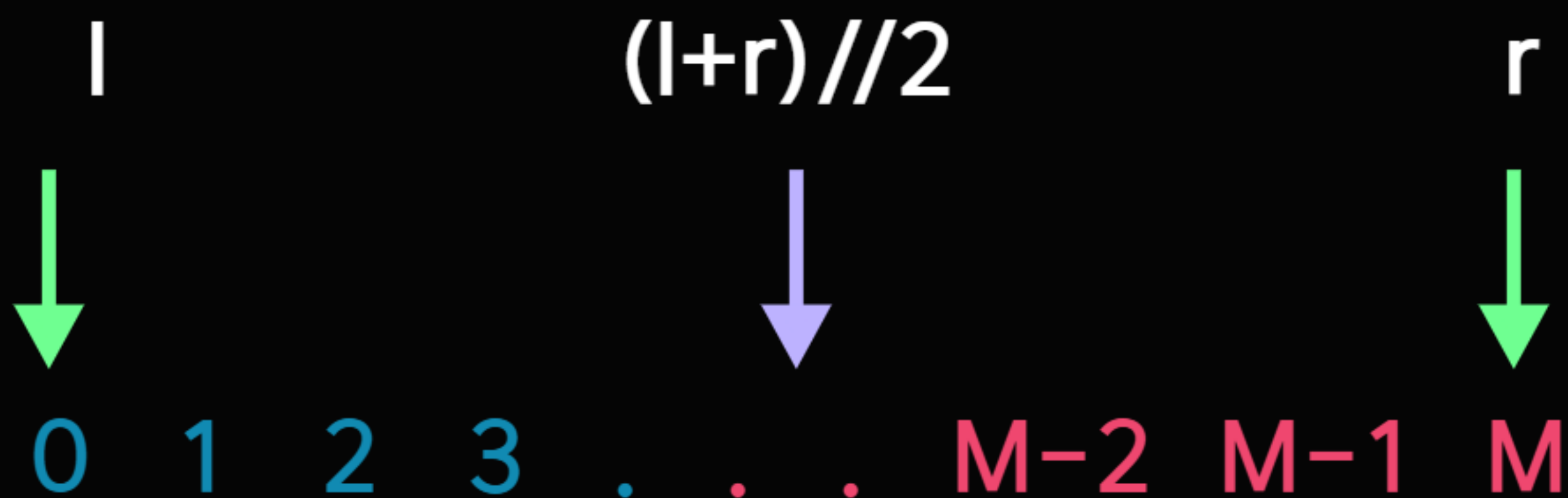
어떤 때에 사용할까?



l을 풍선 M개를 만들 수 없는 시간,
r을 풍선 M개를 만들 수 있는 시간이라고 하자.

02

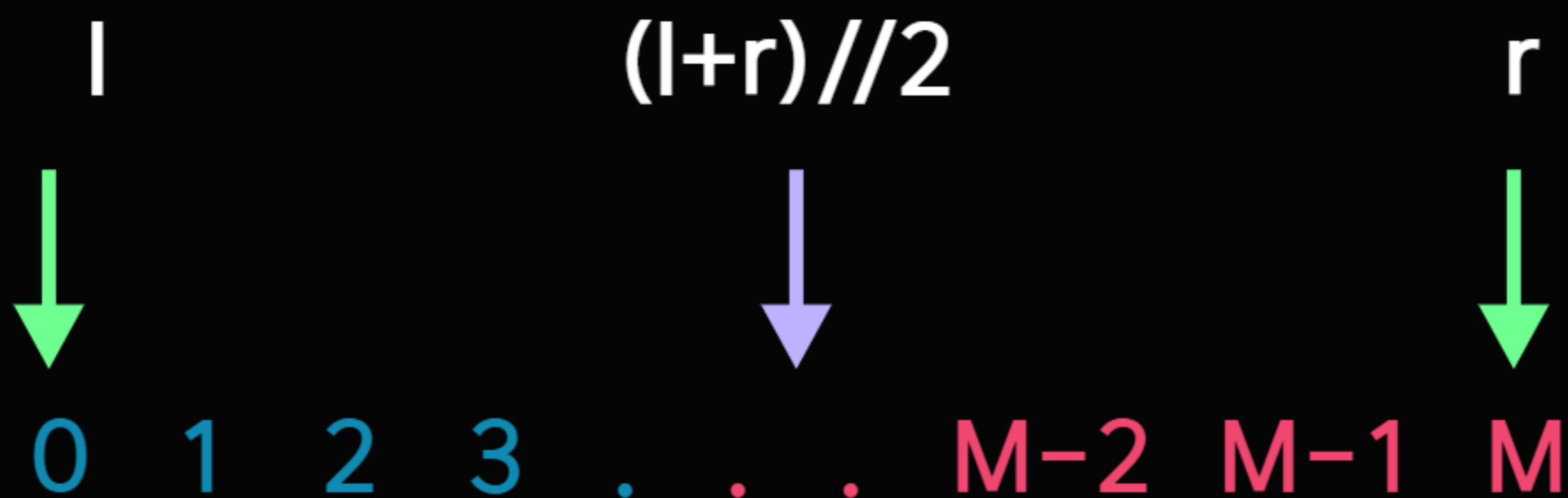
어떤 때에 사용할까?



아까처럼 l 과 r 의 중간 지점 m 을 잡는다.
그리고 $T=m$ 일 때 풍선 M 개를 만드는 것이 가능한지 판별한다.

02

어떤 때에 사용할까?



$O(N)$ 의 시간을 들여 판별한 뒤
M개를 만들 수 있었다면 $r = m$
불가능했다면 $l = m$

02

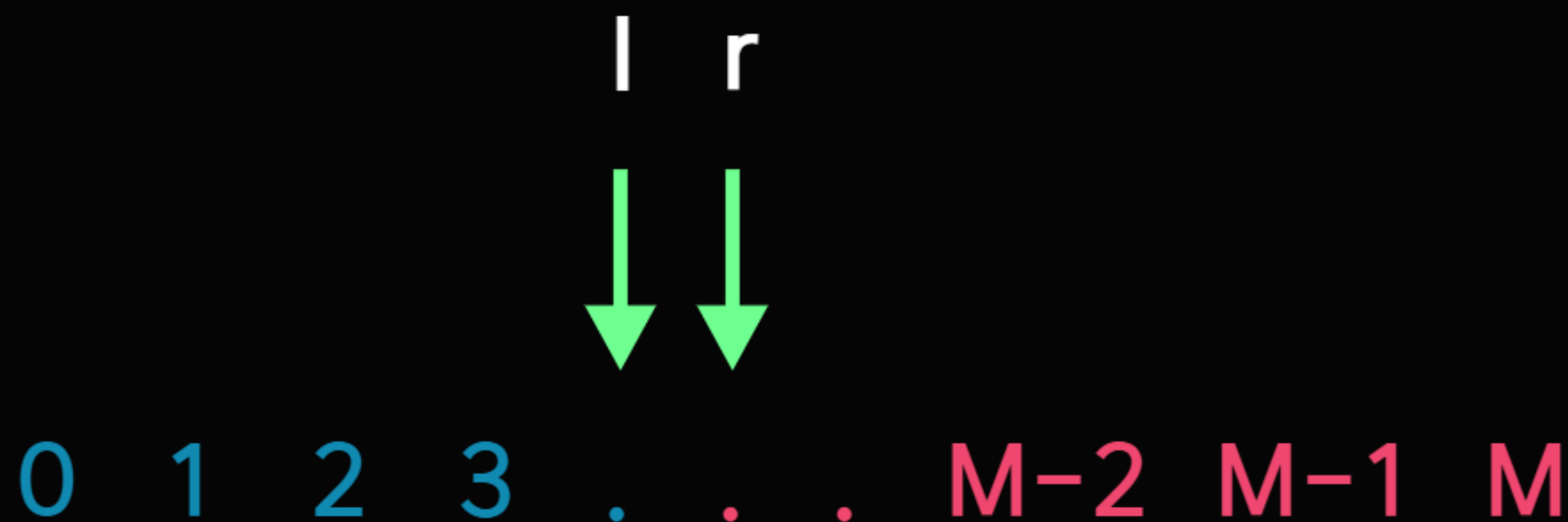
어떤 때에 사용할까?



만약 가능했다면
이런 상태가 될 것이다.
같은 방법으로 $l+1 = r$ 이 될 때까지 반복한다.

02

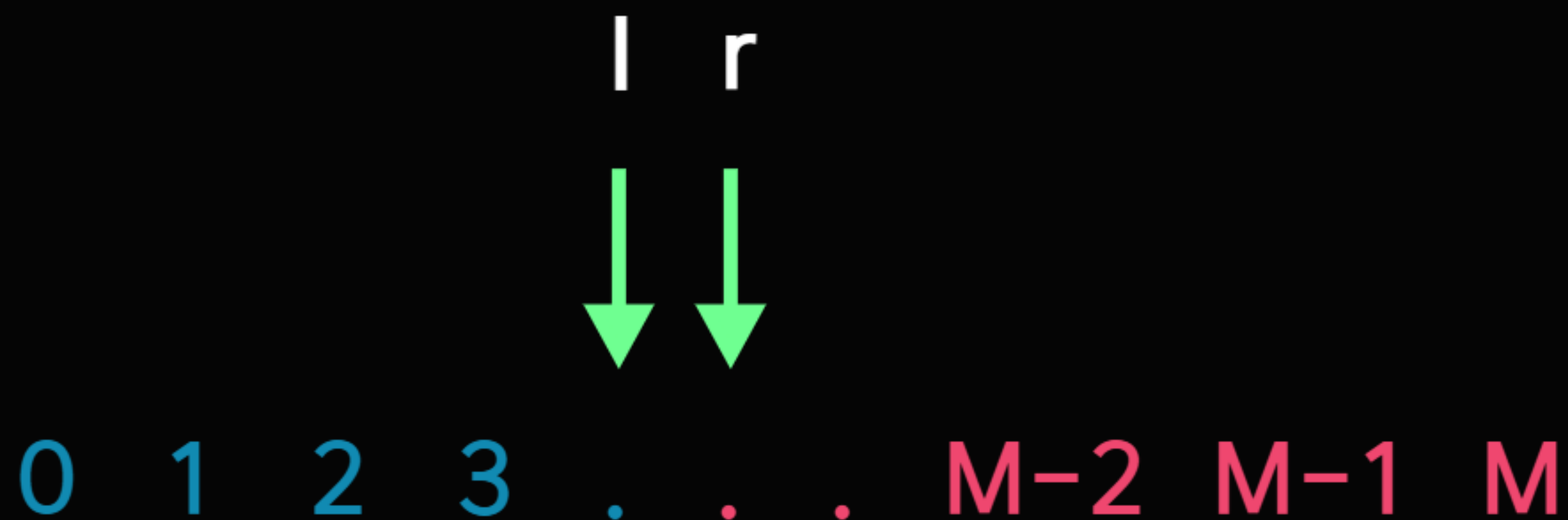
어떤 때에 사용할까?



0과 M 사이 어딘가에서 $l+1 = r$ 인 지점이 있을 것이다.
이때 r 값이 풍선 M개를 만들 수 있는 시간 T의 최솟값이다.

02

어떤 때에 사용할까?



여기서 이분탐색 횟수는 $O(\log M)$,
각 탐색마다 조건을 판별하는 데 $O(N)$ 이 걸렸으므로
 $O(N \log M)$ 에 문제를 해결할 수 있다.

어떻게 이렇게 가능할까?
그 답은 단조성에 있다.

이 문제의 경우 시간 T 가 늘어날수록 만들어지는 풍선의 수는 단조증가한다.
시간은 늘어났지만 만들 수 있는 풍선의 수가 적어지는 경우는
절대 있을 수 없다.

0 1 2 3 4 5 6 7 8 9 10

답이 $T=6$ 인 경우를 생각해보자.

6보다 작은 T 에서는 절대 풍선 M 개를 만들 수 없고,

6보다 큰 T 에서는 반드시 풍선 M 개를 만들 수 있다.

이 성질을 이용하면 이분탐색으로 **조건을 만족하는 값의 경계**를 찾아낼 수 있다.

이런 풀이 방식을 **매개변수 탐색**이라고 한다.

매개변수 탐색을 이용하면
최적화 문제를 결정 문제로 바꿀 수 있다.

최적 값을 계산해서 한번에 찾기 ->

이 값이 조건에 맞는지 아닌지만 판별하기. 최적 값은 이분탐색이 찾아준다



02

어떤 때에 사용할까?

귀찮은 최적화 문제를 뇌를 빼고 풀 수 있게 해주는 아주 좋은 테크닉이다.

어떤 때에 사용할까?

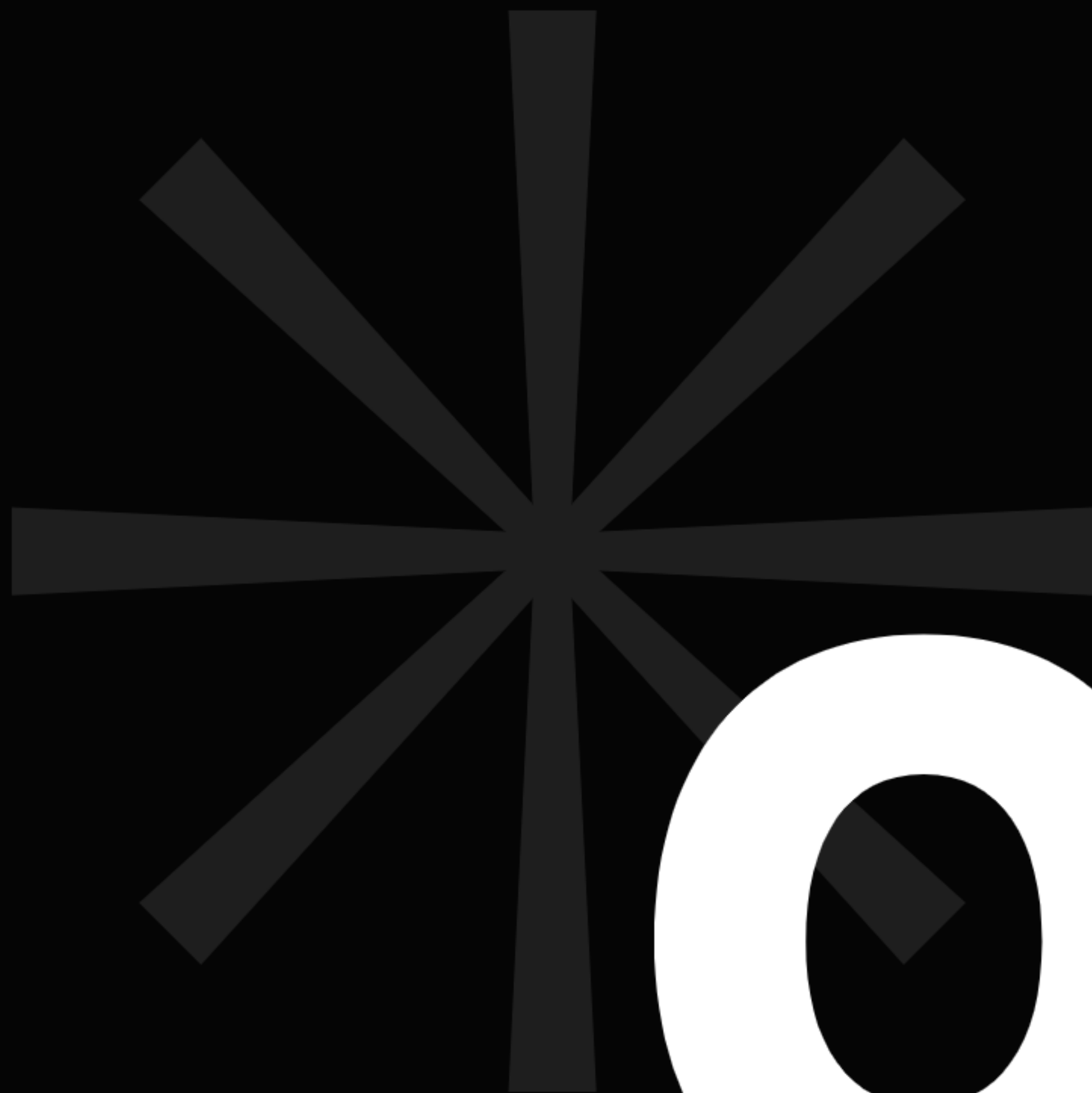
```
import sys
input = sys.stdin.readline

N, M = map(int, input().split())
staff = list(map(int, input().split()))

start, end = 0, 10**12+7
while end > start + 1:
    mid = (start+end)//2
    t = 0
    for i in staff:
        t += mid//i
    if t >= M: # 되거나 큼
        end = mid
    else:
        start = mid

print(end)
```


문제
풀어보기



03



03

문제 풀어보기

예제: 엉성한 도토리 분류기 (#31848)

이 문제에서 어떻게 단조성을 찾아야 할까?
구멍의 크기는 엉망진창으로 나열되어 있는데...

정답은 아주 간단하다.
앞에 크기 N 의 구멍이 있었다면
그 구멍보다 작은 뒷쪽 구멍들은 도토리가 통과할 가능성이 없다.
이런 구멍들을 미리 걸러내면 **오름차순의 수열**이 완성될 것이다.

그런데 문제가 있다.

도토리는 떨어지지 않는 구멍을 통과할 때마다 크기가 1씩 줄어든다.

그러면 구멍을 하나하나 지나치면서 크기를 1씩 줄이는 시뮬레이션을 돌려야 할까?

관점을 바꿔보자.

크기 $a-1$ 의 도토리가 크기 b 의 구멍을 지나치는 상황

크기 a 의 도토리가 크기 $b+1$ 의 구멍을 지나치는 상황

이 둘은 놀랍게도 **같은 상황**으로 간주할 수 있다.

도토리에 비해 상대적인 구멍의 크기는 같기 때문이다.

그렇다면

구멍을 지나칠 때마다 도토리의 크기가 1씩 줄어드는 것과
구멍을 지나칠 때마다 다음 구멍들의 크기가 1씩 늘어나는 것은 본질적으로 같다.

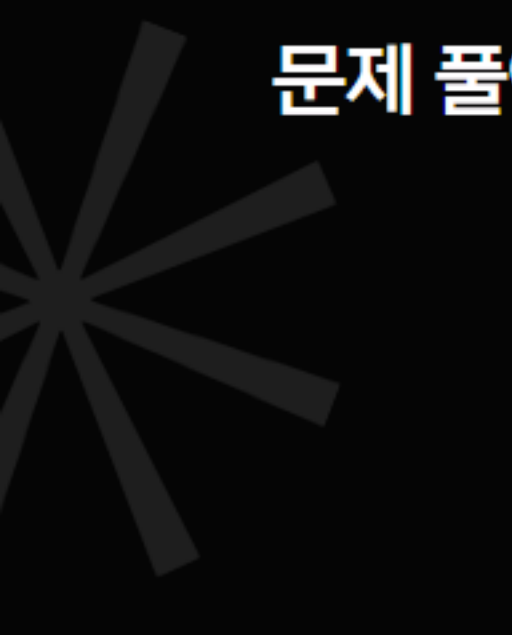
그렇다면 아까 했던 "더 작은 구멍을 삭제"하는 전처리와
"구멍의 크기를 앞서 지나친 구멍의 수만큼 늘리는" 전처리를 동시에 해보자.


```
N = int(input().strip())
hole = [(-1, 0)] # 더미 구멍
idx = 0
for size in map(int, input().split()):
    if hole[-1][0] < size+idx:
        hole.append((size+idx, idx+1))
    idx += 1
```



03

문제 풀어보기



이제 구멍의 전처리는 완료했으니,
도토리를 굴러볼 차례다.

여기서부터는 아주 간단하다.

매개변수 탐색으로

도토리 크기보다 큰 구멍 중 가장 앞에 있는 구멍을 골라주면 된다.



03

문제 풀어보기

그래서, 아까 더미 구멍을 추가한 이유는 뭘까?

매개변수 탐색을 진행해 보자.
l, r의 상태를 각각 **도토리가 빠질 수 없는 상태**와
도토리가 빠질 수 있는 상태로 구분해서 진행하면
빠짐과 안 빠짐의 경계를 찾을 수 있을 것이다.

여기서 더미 구멍을 추가하지 않았다고 가정해 보자.
만약 첫 번째 구멍의 크기가 5고,
도토리 크기가 3이라면 $l=0$ 으로 초기값을 지정해도 정의에 문제가 발생한다.

그래서 절대 도토리가 빠질 수 없는 구멍을 0번 구멍으로 지정해서
더미 데이터를 넣어주었다.
 $(-1, 0)$ 일 때 크기가 음수인 구멍에 도토리가 빠질 수는 없으므로
1의 정의가 유효하다.

더미 구멍을 추가하지 않고 미리 **전처리**하는 방식도 가능하다.

1. 첫번째 구멍부터 빠지는 경우와
2. 마지막 구멍에도 빠지지 않는 경우를 나눠 검사한 뒤
둘 다 아닐 때만 이분 탐색을 진행하면 된다.

하지만 이 문제에서는 도토리가 구멍에 빠지지 않는 경우는 없다고 명시해 뒀으므로
2번 케이스는 제외한다.


```
def solve():
    N = int(input().strip())
    hole = [(-1, 0)]
    idx = 0
    for size in map(int, input().split()):
        if hole[-1][0] < size+idx:
            hole.append((size+idx, idx+1))
        idx += 1

    Q = int(input().strip())
    for i in map(int, input().split()):
        l, r = 0, len(hole)-1
        while l+1 < r:
            m = (l+r)//2
            if hole[m][0] >= i:
                r = m
            else:
                l = m
        print(hole[r][1])
```

이 문제는 도토리의 입력도 전처리해서
 $O(N + Q \log Q + Q)$ 로 풀 수도 있다.

오프라인 쿼리라고도 하는 테크닉인데,
관심이 있는 사람만 아래 코드를 살펴보도록 하자!


```
import sys
import heapq
input = sys.stdin.readline

N = int(input().strip())
hole = list(map(int, input().split()))
for i in range(N):
    hole[i] += i
Q = int(input().strip())
hq = []
dot = list(map(int, input().split()))
dotori = []
for i in range(Q):
    dotori.append((i, dot[i]))
dotori.sort(key=lambda x: x[1])

res = [0]*Q
idx = 0
for i in range(Q):
    while dotori[i][1] > hole[idx]:
        idx += 1
    res[dotori[i][0]] = idx+1
for i in range(Q):
    print(res[i])
```

강의가 끝났으니 밝히는 진실:

사실 파이썬에는 `bisect`라는 이분탐색 모듈이 있다.

이 모듈을 사용하면 배열 내에서 a 보다 처음으로 커지는 인덱스,
처음으로 작아지는 인덱스를 딸깍으로 구할 수 있다. (처음에 예시로 봤던 상황과 똑같다)

하지만 찾고자 하는 값이 배열에 들어가 있어야 한다는 제한사항이 있어서
자주 쓰게 되지는 않는 것 같다.

이분탐색은 굳이 모듈을 쓰지 않아도 구현이 간단한 편이니
이렇게 있다는 것만 알아두고, 외워서 쓸 수 있도록 하자.

강의 설문조사



Thank you For Listening

5주차 스터디를 들어주셔서 감사합니다.
뒷풀이에 참여하실 분들만 강의실에 남아주세요!

—

